

Reporte de Auditoría — AAAA

Análisis estático automatizado mediante Ghidra + Motor de Reglas + NIST NVD

BINARIO

AAAA

FUNCIONES ANALIZADAS

7

GENERADO

29/06/2026 05:05

CLASIFICACIÓN

CONFIDENCIAL

DETECCIONES DEL MOTOR DE REGLAS LOCAL

- Improper Neutralization of Special Elements used in an OS Command** en `FUN_00401020` (Evidencia: `sh`).
Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- NULL Pointer Dereference** en `FUN_00401020` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- NULL Pointer Dereference** en `FUN_00401100` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- Buffer Copy without Checking Size (Classic Buffer Overflow)** en `main` (Evidencia: `scanf`). Mitigación: Usa versiones seguras como `fgets`, `strncpy` o `snprintf`.
- Improper Neutralization of Special Elements used in an OS Command** en `main` (Evidencia: `sh`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- NULL Pointer Dereference** en `main` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- Buffer Copy without Checking Size (Classic Buffer Overflow)** en `iniciar` (Evidencia: `scanf`). Mitigación: Usa versiones seguras como `fgets`, `strncpy` o `snprintf`.
- Improper Neutralization of Special Elements used in an OS Command** en `iniciar` (Evidencia: `sh`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- NULL Pointer Dereference** en `iniciar` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- Use of Externally-Controlled Format String** en `imprimir` (Evidencia: `printf`). Mitigación: Nunca pases variables directamente al primer argumento. Usa `printf("%s", var)`.
- Improper Neutralization of Special Elements used in an OS Command** en `imprimir` (Evidencia: `sh`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- NULL Pointer Dereference** en `imprimir` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- Buffer Copy without Checking Size (Classic Buffer Overflow)** en `enviar` (Evidencia: `scanf`). Mitigación: Usa versiones seguras como `fgets`, `strncpy` o `snprintf`.

- **Use of Externally-Controlled Format String** en `enviar` (Evidencia: `printf`). Mitigación: Nunca pases variables directamente al primer argumento. Usa `printf("%s", var)`.
- **Improper Neutralization of Special Elements used in an OS Command** en `enviar` (Evidencia: `sh`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- **NULL Pointer Dereference** en `enviar` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- **Improper Neutralization of Special Elements used in an OS Command** en `rce` (Evidencia: `system`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- **Improper Neutralization of Special Elements used in an OS Command** en `rce` (Evidencia: `sh`). Mitigación: Evita llamadas al sistema. Usa APIs nativas de la librería estándar.
- **NULL Pointer Dereference** en `rce` (Evidencia: `0x0`). Mitigación: Verifica siempre que el puntero devuelto por funciones como `malloc` no sea `NULL` antes de usarlo.
- **Potential Backdoor / Shell access** en `rce` (Evidencia: `/bin/sh`). Mitigación: Presencia de rutas de shell en el binario. Investiga si se usan en `system()` o `exec()`.

❑ ANEXO: CÓDIGO EXTRAÍDO

Función: `FUN_00401020`

```
void FUN_00401020(void)
{
    (*(code *)PTR_00404010)();
    return;
}
```

Función: `FUN_00401100`

```
/* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
void FUN_00401100(void)
{
    (*_DAT_00404048)();
    return;
}
```

Función: `main`

```

void main(void)
{
    int local_2c;
    undefined local_28 [32];
    setvbuf(stdout, (char *)0x0, 2, 0);
    do {
        puts("Opciones:");
        puts(&DAT_00402012);
        puts("2: Imprimir nombre");
        puts("3: Enviar mensaje");
        puts("4: Salir");
        puts("5: RCE");
        puts(&DAT_00402060);
        ");
        __isoc99_scanf(&DAT_00402089, &local_2c);
        if (local_2c == 4) {
            FUN_00401100(0);
            LAB_004012f9:
            puts("OPCION INCORRECTA");
        }
        else {
            if (4 < local_2c) goto LAB_004012f9;
            if (local_2c == 3) {
                enviar();
            }
            else {
                if (3 < local_2c) goto LAB_004012f9;
                if (local_2c == 1) {
                    iniciar(local_28);
                }
                else {
                    if (local_2c != 2) goto LAB_004012f9;
                    imprimir(local_28);
                }
            }
        }
        putchar(10);
    } while( true );
}

```

Función: `iniciar`

```

void iniciar(undefined8 param_1)
{
    ");
    __isoc99_scanf(&DAT_004020c2, param_1);
    return;
}

```

Función: `imprimir`

```

void imprimir(undefined8 param_1)
{
    printf("\nTu nombre es: %s\n", param_1);
    return;
}

```

Función: `enviar`

```
void enviar(void)
{
    undefined local_f8 [208];
    undefined local_28 [32];
    printf(&DAT_004020e0);
    __isoc99_scanf(&DAT_004020c2,local_28);
    puts(&DAT_00402108);
    __isoc99_scanf(&DAT_0040212a,local_f8);
    printf("Mensaje enviado a %s",local_28);
    return;
}
```

Función: `rce`

```
void rce(void)
{
    system("/bin/sh");
    return;
}
```